

Node.js Tutorial

A Complete Guide

15 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:
<https://snehaliteng.github.io/tutorials/nodejs/>

Getting Started with Node.js

What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine. It allows you to run JavaScript on the server side, enabling you to build fast, scalable network applications.

Key Benefit: Node.js uses non-blocking, event-driven I/O, making it lightweight and efficient for data-intensive real-time applications.

Why Choose Node.js?

- **JavaScript Everywhere:** Use the same language for frontend and backend.
- **High Performance:** Built on V8, which compiles JS to native machine code.
- **Non-blocking I/O:** Handles thousands of concurrent connections with a single thread.
- **Rich Ecosystem:** NPM provides over 2 million packages.
- **Active Community:** Backed by OpenJS Foundation and major companies.

System Requirements

- **Operating System:** Linux, macOS, or Windows
- **Memory:** Minimum 512 MB RAM (1 GB recommended)
- **Disk Space:** At least 200 MB for Node.js and NPM
- **Node.js Version:** LTS (Long Term Support) recommended

Node.js vs Browser JavaScript

Aspect	Browser	Node.js
Global Object	window	global
DOM/BOM	Available	Not available
Module System	ES Modules (import/export)	CommonJS + ES Modules
File System	No access	Full access (fs module)
Runtime	Sandboxed	Full system access

Command Line Basics

Node.js is primarily used through the command line. Here are essential commands:

```
# Check Node.js version
node --version

# Run a JavaScript file
node app.js

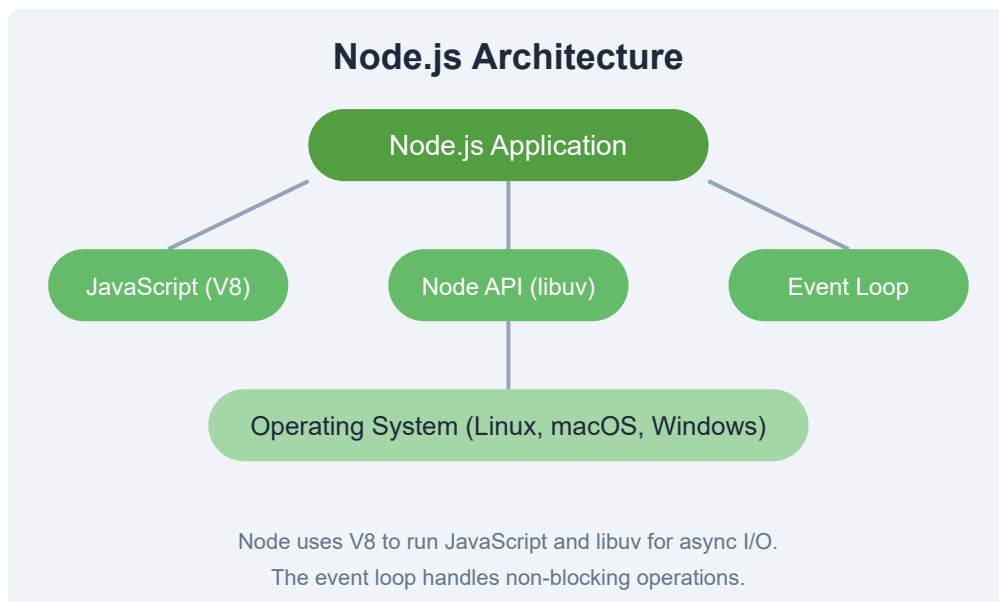
# Start REPL (interactive shell)
node

# Run with inspector for debugging
node --inspect app.js

# Execute a one-liner
node -e "console.log('Hello, Node!')"
```

The V8 Engine

V8 is Google's open-source high-performance JavaScript engine, written in C++. It compiles JavaScript directly to native machine code using JIT (Just-In-Time) compilation. Node.js embeds V8 to execute JavaScript on the server.



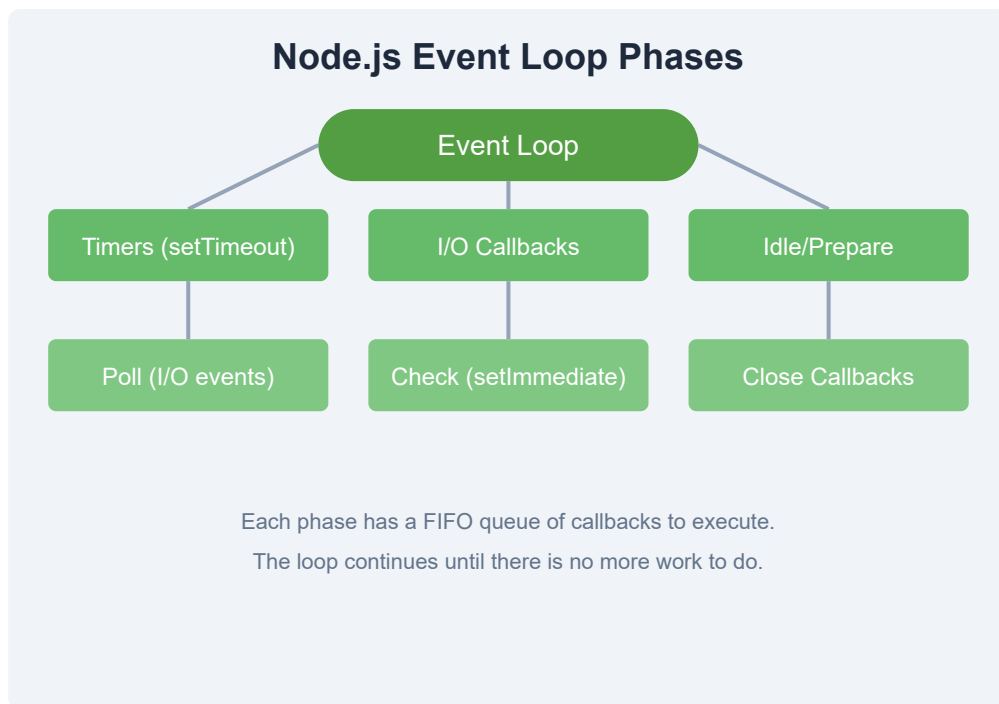
Node.js Architecture

Node.js consists of three main layers:

1. **JavaScript (V8):** Handles your application code and core JS execution.
2. **Node API (libuv):** Provides asynchronous I/O operations like file system, networking, and concurrency.
3. **Event Loop:** Coordinates callbacks and ensures non-blocking execution.

The Event Loop

The event loop is what allows Node.js to perform non-blocking I/O operations despite JavaScript being single-threaded. It works by offloading operations to the system kernel whenever possible.



The event loop has several phases:

- **Timers:** Executes callbacks scheduled by `setTimeout()` and `setInterval()`.
- **I/O Callbacks:** Executes almost all callbacks except timers, close, and `setImmediate`.
- **Idle/Prepare:** Used internally by Node.js.
- **Poll:** Retrieves new I/O events and executes their callbacks.
- **Check:** Executes `setImmediate()` callbacks.
- **Close Callbacks:** Executes close event callbacks (e.g., `socket.on('close')`).

Tip: Understanding the event loop is crucial for writing performant Node.js applications. Always avoid blocking the event loop with synchronous operations.

Prerequisites

Before diving deeper, ensure you have:

- Basic knowledge of JavaScript (variables, functions, loops, objects)
- Familiarity with command line / terminal
- Node.js installed (download from nodejs.org)
- A code editor (VS Code recommended)

Your First Node.js Application

Create a file named `hello.js` and add:

```
// hello.js
console.log("Hello, Node.js!");

// Access process info
console.log("Node version:", process.version);
console.log("Platform:", process.platform);
console.log("Current directory:", process.cwd());
```

Run it with: `node hello.js`

Exercise: Create a Node.js script that prints your name, the current date and time, and the operating system platform. Run it from the terminal.

Asynchronous Programming in Node.js

Why Asynchronous Programming?

Node.js excels at asynchronous programming because it's built on a non-blocking I/O model. This means while one operation is waiting (e.g., reading a file or making a network request), the event loop can handle other tasks. Understanding async patterns is essential for writing efficient Node.js code.

Callbacks

Callbacks are the original async pattern in Node.js. A callback is a function passed as an argument to another function that gets called when the operation completes.

```
const fs = require('fs');

// Asynchronous read with callback
fs.readFile('data.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Error reading file:', err);
    return;
  }
  console.log('File content:', data);
});

console.log('Reading file...'); // This runs BEFORE the callback
```

Common Convention: Node.js callbacks follow the "error-first" pattern — the first parameter is an error object (or null if successful), and subsequent parameters contain the result.

Callback Hell

Nesting callbacks leads to deeply indented code that's hard to read and maintain:

```
fs.readdir('folder', (err, files) => {
  if (err) throw err;
  fs.readFile(files[0], 'utf8', (err, data) => {
    if (err) throw err;
    fs.writeFile('output.txt', data, (err) => {
      if (err) throw err;
      console.log('Done!');
    });
  });
});
```

Promises

Promises provide a cleaner way to handle asynchronous operations. A Promise represents a value that may be available now, in the future, or never.

```
const fs = require('fs').promises;

// Asynchronous read with Promise
fs.readFile('data.txt', 'utf8')
  .then(data => {
    console.log('File content:', data);
    return fs.writeFile('output.txt', data);
  })
  .then(() => console.log('Successfully wrote to output.txt'))
  .catch(err => console.error('Error:', err));

console.log('Reading file...'); // This still runs first
```

Creating a Promise

```
function delay(ms) {
  return new Promise((resolve, reject) => {
    if (ms < 0) {
      reject(new Error('Delay must be positive'));
    } else {
      setTimeout(() => resolve(`Delayed by ${ms}ms`), ms);
    }
  });
}

delay(1000)
  .then(message => console.log(message))
  .catch(err => console.error(err));
```

Async/Await

Async/await is syntactic sugar over promises, making asynchronous code read like synchronous code. It's the recommended approach in modern Node.js.

```
const fs = require('fs').promises;

async function processFiles() {
  try {
    const data = await fs.readFile('data.txt', 'utf8');
    console.log('File content:', data);
    await fs.writeFile('output.txt', data);
    console.log('Successfully wrote to output.txt');
  } catch (err) {
    console.error('Error:', err);
  }
}

processFiles();
```

Parallel Execution

```
async function readMultipleFiles() {
  const files = ['file1.txt', 'file2.txt', 'file3.txt'];
  const promises = files.map(f => fs.readFile(f, 'utf8'));
  const results = await Promise.all(promises);
  return results;
}

readMultipleFiles().then(console.log);
```

Error Handling in Async Code

- **Callbacks:** Check the `err` parameter in the callback.
- **Promises:** Use `.catch()` at the end of the chain.
- **Async/Await:** Wrap with `try/catch` blocks.

```
// Global unhandled promise rejection handler
process.on('unhandledRejection', (reason, promise) => {
  console.error('Unhandled Rejection at:', promise, 'reason:', reason);
});
```

Exercise: Write an async function that reads three files in parallel, concatenates their contents, and writes the result to a new file. Handle errors appropriately.

Modules & Package Management

CommonJS Modules

CommonJS is the original module system in Node.js. It uses `require()` to import modules and `module.exports` to export them.

```
// math.js (export)
function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }
module.exports = { add, subtract };

// app.js (import)
const math = require('./math');
console.log(math.add(5, 3)); // 8
console.log(math.subtract(10, 4)); // 6
```

Caching: Modules are cached after the first `require()` call. Subsequent calls return the same instance.

ES Modules

ES Modules (ESM) are the official JavaScript standard. Use `import` and `export` with a `"type": "module"` in `package.json` or the `.mjs` extension.

```
// utils.mjs (export)
export function greet(name) {
  return `Hello, ${name}!`;
}
export const PI = 3.14159;

// main.mjs (import)
import { greet, PI } from './utils.mjs';
console.log(greet('Node'));
console.log('PI:', PI);
```

package.json Configuration for ESM

```
{
  "name": "my-app",
  "version": "1.0.0",
  "type": "module",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  }
}
```

NPM Basics

NPM (Node Package Manager) is the default package manager for Node.js. It's used to install, manage, and share packages.

```
# Initialize a new project (creates package.json)
npm init
npm init -y # Skip prompts

# Install packages
npm install express
npm install --save-dev nodemon
npm install lodash@4.17.21 # Specific version

# Install globally
npm install -g typescript

# Uninstall packages
npm uninstall lodash

# List installed packages
npm list --depth=0
```

Package.json in Detail

The `package.json` file is the heart of any Node.js project. It contains metadata, dependencies, scripts, and configuration.

```
{
  "name": "my-awesome-app",
  "version": "2.1.0",
  "description": "A sample Node.js application",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js",
    "dev": "nodemon src/index.js",
    "test": "jest",
    "build": "tsc",
    "lint": "eslint ."
  },
  "dependencies": {
    "express": "^4.18.0",
    "mongoose": "^7.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0",
    "jest": "^29.0.0",
    "eslint": "^8.0.0"
  },
  "author": "Your Name",
  "license": "MIT"
}
```

NPM Scripts

NPM scripts run commands defined in `package.json`. They're a convenient way to automate common tasks.

```
npm run dev      # Run "dev" script
npm run test     # Run "test" script
npm start        # Runs "start" script
npm test         # Runs "test" script
npm run build    # Run "build" script
```

Pre/Post Hooks: NPM automatically runs `pre` and `post` scripts if defined. For example, `prestart` runs before `start`.

Dependency Management

- **dependencies:** Required at runtime (express, lodash, mongoose).
- **devDependencies:** Only needed during development (testing, linting, build tools).
- **peerDependencies:** Plugins that need a compatible version of a host package.
- **package-lock.json:** Locks exact versions for reproducible builds.

Semantic Versioning (SemVer)

NPM uses semantic versioning: `MAJOR.MINOR.PATCH`

- **^1.2.3:** Compatible with version 1.x.x
- **~1.2.3:** Compatible with version 1.2.x
- **1.2.3:** Exact version
- *****: Any version (not recommended)

Publishing Packages

```
# Login to NPM
npm login

# Build your package
npm run build

# Publish
npm publish

# Update version
npm version patch # 1.0.0 -> 1.0.1
npm version minor # 1.0.1 -> 1.1.0
npm version major # 1.1.0 -> 2.0.0
```

Exercise: Create a new Node.js project, install the `chalk` package (for colored terminal output), and write a script that prints "Hello, NPM!" in green.

Node.js Core Modules

Node.js provides a rich set of built-in modules that don't require any installation. Here are the most commonly used ones.

HTTP/HTTPS

The `http` and `https` modules allow you to create web servers and make HTTP requests.

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello, World!\n');
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

File System (fs)

The `fs` module provides an API for interacting with the file system.

```
const fs = require('fs');
const fsPromises = require('fs').promises;

// Read file synchronously
const data = fs.readFileSync('file.txt', 'utf8');

// Read file asynchronously with callback
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});

// Read file with promises (recommended)
async function read() {
  const data = await fsPromises.readFile('file.txt', 'utf8');
  console.log(data);
}

// Write to file
fs.writeFileSync('output.txt', 'Hello, Node!');

// Directory operations
fs.mkdirSync('new-folder', { recursive: true });
fs.readdirSync('.').forEach(file => console.log(file));
```

Path

The `path` module provides utilities for working with file and directory paths.

```

const path = require('path');

const filePath = '/users/node/app.js';

console.log(path.basename(filePath));    // app.js
console.log(path.dirname(filePath));     // /users/node
console.log(path.extname(filePath));     // .js
console.log(path.parse(filePath));       // { root, dir, base, ext, name }

// Join paths safely
const fullPath = path.join(__dirname, 'public', 'index.html');
console.log(fullPath);

// Resolve to absolute path
console.log(path.resolve('src', 'index.js'));

```

OS

The `os` module provides operating system-related utility methods.

```

const os = require('os');

console.log('Platform:', os.platform());
console.log('Architecture:', os.arch());
console.log('CPUs:', os.cpus().length);
console.log('Total Memory:', os.totalmem());
console.log('Free Memory:', os.freemem());
console.log('Home Dir:', os.homedir());
console.log('Hostname:', os.hostname());
console.log('Network Interfaces:', os.networkInterfaces());

```

URL

```
const url = require('url');

const myUrl = new URL('https://example.com:8080/path?name=node&version=18');
console.log('Protocol:', myUrl.protocol);
console.log('Host:', myUrl.host);
console.log('Pathname:', myUrl.pathname);
console.log('Query Params:', myUrl.searchParams.get('name'));

// Format URL
console.log(myUrl.toString());
```

Events

Node.js has a built-in event system that many modules use. The `events` module provides the `EventEmitter` class.

```
const EventEmitter = require('events');

class MyEmitter extends EventEmitter {}
const myEmitter = new MyEmitter();

myEmitter.on('customEvent', (arg) => {
  console.log('Event fired with:', arg);
});

myEmitter.emit('customEvent', { data: 'Hello' });
```

Streams

Streams are used to handle large amounts of data efficiently by processing chunks instead of loading everything into memory.

```

const fs = require('fs');
const zlib = require('zlib');

// Readable stream
const readStream = fs.createReadStream('input.txt', 'utf8');
// Writable stream
const writeStream = fs.createWriteStream('output.txt');

// Pipe data from read stream to write stream
readStream.pipe(writeStream);

// Chain transforms (compress)
fs.createReadStream('input.txt')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('input.txt.gz'));

```

Buffers

Buffers handle raw binary data in Node.js.

```

// Create a buffer
const buf = Buffer.from('Hello, Node!');
console.log(buf);           //
console.log(buf.toString()); // Hello, Node!
console.log(buf.length);    // 12

// Buffer operations
const buf2 = Buffer.alloc(10);
buf2.write('Hi');
console.log(buf2.toString()); // Hi

// Concatenate buffers
const combined = Buffer.concat([buf, Buffer.from(' World')]);
console.log(combined.toString()); // Hello, Node! World

```

Crypto

```
const crypto = require('crypto');

// Create a hash
const hash = crypto.createHash('sha256')
  .update('Hello, Node!')
  .digest('hex');
console.log('Hash:', hash);

// Generate random bytes
const randomBytes = crypto.randomBytes(16);
console.log('Random:', randomBytes.toString('hex'));

// Create HMAC
const hmac = crypto.createHmac('sha256', 'secret-key')
  .update('message')
  .digest('hex');
```

Other Notable Core Modules

- **Timers:** `setTimeout`, `setInterval`, `setImmediate`
- **DNS:** Lookup domain names, resolve records
- **Assert:** Write tests with assertion statements
- **Util:** Utility functions like `promisify`, `inspect`
- **Readline:** Read input from a readable stream line by line
- **Zlib:** Compression and decompression

Exercise: Build a simple CLI tool that reads a file, compresses it with `zlib`, calculates the SHA-256 hash of the original content, logs the current memory usage and CPU count, and writes the hash to a metadata file.

JavaScript & TypeScript Features

Modern JavaScript Features

Node.js supports the latest ECMAScript features. Here are some essential ones:

Destructuring

```
// Object destructuring
const person = { name: 'Alice', age: 30, city: 'NYC' };
const { name, age } = person;
console.log(name, age);

// Array destructuring
const colors = ['red', 'green', 'blue'];
const [first, ...rest] = colors;
console.log(first, rest);
```

Spread & Rest Operators

```
// Spread (expand arrays/objects)
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];
console.log(arr2); // [1, 2, 3, 4, 5]

const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 };
console.log(obj2); // { a: 1, b: 2, c: 3 }

// Rest (collect remaining arguments)
function sum(...nums) {
  return nums.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4)); // 10
```

Arrow Functions & Template Literals

```
// Arrow functions
const double = x => x * 2;
const greet = name => `Hello, ${name}!`;

// Template literals
const user = 'Node';
const version = 18;
console.log(`Running ${user} version ${version}`);
```

Optional Chaining & Nullish Coalescing

```
const user = { profile: { name: 'Alice' } };

// Optional chaining - no error if intermediate is null/undefined
console.log(user?.profile?.name); // Alice
console.log(user?.address?.city); // undefined

// Nullish coalescing - use default only for null/undefined
const port = process.env.PORT ?? 3000;
```

The Process Object

`process` is a global object with information about the current Node.js process.

```
// Environment variables
console.log(process.env.NODE_ENV);
console.log(process.env.PATH);

// Command line arguments
console.log(process.argv);

// Exit process
process.exit(0);    // Success
process.exit(1);    // Error

// Process events
process.on('exit', (code) => {
  console.log(`Process exiting with code ${code}`);
});

process.on('uncaughtException', (err) => {
  console.error('Uncaught exception:', err);
  process.exit(1);
});

// Current working directory
console.log(process.cwd());
```

TypeScript Basics

TypeScript is a typed superset of JavaScript that compiles to plain JavaScript. It's widely used in the Node.js ecosystem.

```
// Install: npm install -g typescript
// Or: npx tsc

// Basic types
let name: string = 'Node';
let version: number = 18;
let isActive: boolean = true;
let data: any = 'could be anything';
let nullable: string | null = null;

// Interfaces
interface User {
  id: number;
  name: string;
  email?: string; // Optional property
}

function getUser(id: number): User {
  return { id, name: 'Alice' };
}
```

TypeScript Configuration

```
// tsconfig.json
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "CommonJS",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules"]
}
```

Advanced TypeScript

```
// Generics
function wrap<T>(value: T): T[] {
  return [value];
}

const wrapped = wrap(42); // number[]
const wrappedStr = wrap('hi'); // string[]

// Utility types
interface Todo {
  title: string;
  completed: boolean;
}

const partial: Partial<Todo> = { title: 'Learn Node' };
const readonly: Readonly<Todo> = { title: 'Done', completed: true };
// readonly.title = 'Try'; // Error!

// Type guards
function isString(val: unknown): val is string {
  return typeof val === 'string';
}
```

Linting & Formatting

```
// ESLint configuration (.eslintrc.json)
{
  "env": { "node": true, "es2022": true },
  "extends": ["eslint:recommended"],
  "parserOptions": { "ecmaVersion": "latest" },
  "rules": {
    "no-unused-vars": "warn",
    "no-console": "off"
  }
}

// Prettier configuration (.prettierrc)
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5"
}
```

Exercise: Create a TypeScript file that defines an interface for a "Task" (id, title, completed, dueDate), implements a function to filter incomplete tasks, and logs them. Compile and run with `npx tsc`.

Building Applications with Node.js

Frameworks Overview

While Node.js core provides the HTTP module, frameworks simplify building web applications:

- **Express.js:** Minimal, flexible, most popular
- **Koa:** Modern, leverages async/await
- **Fastify:** Fast, schema-based validation
- **NestJS:** Structured, opinionated (like Angular)
- **Hapi:** Configuration-driven

Express.js

Express is the de facto standard web framework for Node.js. It provides a robust set of features for web and mobile applications.

```
// Install: npm install express

const express = require('express');
const app = express();
const port = 3000;

// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Routes
app.get('/', (req, res) => {
  res.send('Hello, Express!');
});

app.get('/api/users', (req, res) => {
  res.json([
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' }
  ]);
});

app.post('/api/users', (req, res) => {
  const user = req.body;
  res.status(201).json({ message: 'User created', user });
});

app.listen(port, () => {
  console.log(`Server listening at http://localhost:${port}`);
});
```

Middleware

Middleware functions are the building blocks of Express applications. They have access to the request object, response object, and the next function.

```

// Custom middleware
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);
  next();
};

const authenticate = (req, res, next) => {
  const token = req.headers.authorization;
  if (!token) {
    return res.status(401).json({ error: 'No token provided' });
  }
  req.user = { id: 1, name: 'Alice' }; // Simplified
  next();
};

// Apply middleware
app.use(logger);
app.use('/api/protected', authenticate);

// Route-level middleware
const validateUser = (req, res, next) => {
  if (!req.body.name) {
    return res.status(400).json({ error: 'Name is required' });
  }
  next();
};

app.post('/api/users', validateUser, (req, res) => {
  res.json({ success: true });
});

```

REST API Design

Method	Route	Action
GET	/api/tasks	List all tasks
GET	/api/tasks/:id	Get a single task
POST	/api/tasks	Create a task
PUT	/api/tasks/:id	Update a task
DELETE	/api/tasks/:id	Delete a task

Authentication

```
// Install: npm install jsonwebtoken bcrypt

const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

const SECRET = process.env.JWT_SECRET || 'fallback-secret';

// Hash password
async function hashPassword(password) {
  const salt = await bcrypt.genSalt(10);
  return bcrypt.hash(password, salt);
}

// Generate JWT
function generateToken(user) {
  return jwt.sign({ id: user.id, email: user.email }, SECRET, {
    expiresIn: '7d'
  });
}

// Verify JWT middleware
function verifyToken(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Access denied' });

  try {
    const decoded = jwt.verify(token, SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    res.status(403).json({ error: 'Invalid token' });
  }
}
```

Integrating with Frontend

Serve static files and enable CORS for your frontend application:

```
// Serve static files
app.use(express.static('public'));

// Enable CORS (Install: npm install cors)
const cors = require('cors');
app.use(cors({ origin: 'http://localhost:5173' }));

// Fallback for SPA routing
app.get('*', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'index.html'));
});
```

Exercise: Build a simple task management REST API using Express.js with the following endpoints: GET /tasks, POST /tasks, PUT /tasks/:id, DELETE /tasks/:id. Use an in-memory array as storage and add basic request validation.

Database Integration

MySQL with Node.js

MySQL is one of the most popular relational databases. Use the `mysql2` package to connect to MySQL from Node.js.

```

// Install: npm install mysql2

const mysql = require('mysql2/promise');

async function connectMySQL() {
  const connection = await mysql.createConnection({
    host: process.env.DB_HOST || 'localhost',
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD || '',
    database: 'node_tutorial'
  });

  // Create table
  await connection.execute(`
    CREATE TABLE IF NOT EXISTS users (
      id INT AUTO_INCREMENT PRIMARY KEY,
      name VARCHAR(100) NOT NULL,
      email VARCHAR(100) UNIQUE NOT NULL,
      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
  `);

  // Insert
  const [result] = await connection.execute(
    'INSERT INTO users (name, email) VALUES (?, ?)',
    ['Alice', 'alice@example.com']
  );
  console.log('Inserted ID:', result.insertId);

  // Query
  const [rows] = await connection.execute('SELECT * FROM users WHERE id = ?',
  console.log('User:', rows[0]));

  // Update
  await connection.execute('UPDATE users SET name = ? WHERE id = ?', ['Alice

  // Delete
  await connection.execute('DELETE FROM users WHERE id = ?', [1]);

  await connection.end();
}

connectMySQL().catch(console.error);

```

MongoDB with Node.js

MongoDB is a popular NoSQL database. Use the official `mongodb` driver or the `mongoose` ODM.

Using Mongoose (ODM)

```
// Install: npm install mongoose

const mongoose = require('mongoose');

// Connect
mongoose.connect('mongodb://localhost:27017/node_tutorial');

// Define schema
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: { type: Number, min: 0 },
  address: {
    street: String,
    city: String
  },
  createdAt: { type: Date, default: Date.now }
});

// Create model
const User = mongoose.model('User', userSchema);

// CRUD operations
async function crudExample() {
  // Create
  const user = await User.create({
    name: 'Bob',
    email: 'bob@example.com',
    age: 25
  });

  // Read
  const found = await User.findOne({ email: 'bob@example.com' });
  console.log(found);

  // Update
  await User.updateOne({ _id: user._id }, { $set: { age: 26 } });

  // Delete
  await User.deleteOne({ _id: user._id });
}
```

MongoDB Aggregations

```
// Aggregation pipeline
const results = await User.aggregate([
  { $match: { age: { $gte: 18 } } },
  { $group: { _id: '$address.city', count: { $sum: 1 }, avgAge: { $avg: '$age' } } },
  { $sort: { count: -1 } },
  { $limit: 10 }
]);
```

Connection Pooling

Always use connection pooling in production to reuse database connections:

```
// MySQL pool
const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  database: 'node_tutorial',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

// Use pool
const [rows] = await pool.execute('SELECT * FROM users');
```

Choosing Between SQL and NoSQL

When to use SQL	When to use NoSQL
Structured, relational data	Flexible, schema-less data
Complex joins and transactions	High volume, simple queries
Data integrity is critical	Rapid prototyping, iterations
ACID compliance required	Horizontal scaling needed

Exercise: Create a product catalog API with Express that connects to MongoDB using Mongoose. Define a Product schema (name, price, category, stock), implement CRUD endpoints, and add an aggregation that groups products by category.

Advanced Communication

GraphQL

GraphQL is a query language for APIs that allows clients to request exactly the data they need. Use `express-graphql` and `graphql` packages to add GraphQL to your Express server.

```

// Install: npm install express-graphql graphql

const { buildSchema } = require('graphql');
const { graphqlHTTP } = require('express-graphql');

// Define schema
const schema = buildSchema(`
  type User {
    id: ID!
    name: String!
    email: String!
    posts: [Post]
  }

  type Post {
    id: ID!
    title: String!
    content: String
  }

  type Query {
    user(id: ID!): User
    users: [User]
    posts: [Post]
  }

  type Mutation {
    createUser(name: String!, email: String!): User
    createPost(title: String!, content: String): Post
  }
`);

// Resolvers
const root = {
  users: () => [
    { id: '1', name: 'Alice', email: 'alice@example.com' }
  ],
  user: ({ id }) => ({
    id, name: 'Alice', email: 'alice@example.com'
  })
};

app.use('/graphql', graphqlHTTP({
  schema,

```

```
    rootValue: root,  
    graphiql: true // Enable GraphQL IDE  
  }));
```

Socket.IO

Socket.IO enables real-time, bidirectional event-based communication between web clients and servers.

```

// Install: npm install socket.io

const http = require('http');
const { Server } = require('socket.io');

const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: '*' }
});

io.on('connection', (socket) => {
  console.log('A user connected:', socket.id);

  // Listen for events
  socket.on('chat message', (msg) => {
    console.log('Message received:', msg);
    // Broadcast to all connected clients
    io.emit('chat message', {
      user: socket.id,
      message: msg,
      timestamp: new Date()
    });
  });

  socket.on('join room', (room) => {
    socket.join(room);
    socket.to(room).emit('notification', `${socket.id} joined ${room}`);
  });

  socket.on('disconnect', () => {
    console.log('User disconnected:', socket.id);
  });
});

server.listen(3000, () => {
  console.log('Socket.IO server running on port 3000');
});

```

Client-Side Socket.IO

```
const socket = io('http://localhost:3000');

// Send message
socket.emit('chat message', 'Hello everyone!');

// Listen for messages
socket.on('chat message', (data) => {
  console.log(`${data.user}: ${data.message}`);
});
```

WebSockets (Native)

Node.js also supports raw WebSockets via the `ws` library for lower-level control.

```
// Install: npm install ws

const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws) => {
  console.log('WebSocket client connected');

  ws.on('message', (message) => {
    console.log('Received:', message.toString());

    // Broadcast to all clients
    wss.clients.forEach((client) => {
      if (client.readyState === WebSocket.OPEN) {
        client.send(`Echo: ${message}`);
      }
    });
  });

  ws.send('Welcome to the WebSocket server!');
});
```

When to use what? Use Socket.IO for higher-level features like rooms, namespaces, and automatic reconnection. Use raw WebSockets (`ws`) when you need minimal overhead and full control.

Polling vs WebSockets

Feature	HTTP Polling	WebSockets
Connection	Opens and closes each request	Persistent open connection
Overhead	High (HTTP headers each time)	Low (minimal framing)
Real-time	Near real-time (depends on interval)	True real-time

Exercise: Build a real-time chat application with Socket.IO that supports multiple chat rooms. Users should be able to join a room, send messages visible only to that room, and see who is typing.

Testing & Debugging

Debugging Techniques

Node.js provides several built-in debugging options:

Console Debugging

```
// Basic console logging
console.log('Debug value:', variable);
console.error('Error:', error);
console.warn('Warning:', warning);
console.table([
  { name: 'Alice', age: 30 },
  { name: 'Bob', age: 25 }
]);

// Console groups
console.group('User Details');
console.log('Name: Alice');
console.log('Age: 30');
console.groupEnd();
```

Using the Built-in Inspector

```
# Start with inspector
node --inspect app.js

# Pause at start
node --inspect-brk app.js

# Open chrome://inspect in Chrome to debug
```

Using debugger statement

```
function calculateTotal(items) {  
  let total = 0;  
  for (const item of items) {  
    debugger; // Execution pauses here  
    total += item.price * item.quantity;  
  }  
  return total;  
}
```

Writing Tests

The built-in `assert` module provides simple assertion testing:

```
const assert = require('assert');  
  
function add(a, b) { return a + b; }  
  
assert.strictEqual(add(2, 3), 5);  
assert.strictEqual(add(-1, 1), 0);  
// assert.strictEqual(add(2, 2), 5); // Would throw AssertionError
```

Test Frameworks

Jest

Jest is the most popular testing framework. Install with: `npm install --save-dev jest`

```
// math.js
function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }
module.exports = { add, subtract };

// math.test.js
const { add, subtract } = require('./math');

describe('Math functions', () => {
  test('adds 1 + 2 to equal 3', () => {
    expect(add(1, 2)).toBe(3);
  });

  test('subtracts 5 - 3 to equal 2', () => {
    expect(subtract(5, 3)).toBe(2);
  });

  test('handles negative numbers', () => {
    expect(add(-1, -2)).toBe(-3);
    expect(subtract(-5, -3)).toBe(-2);
  });
});
```

Mocha & Chai

```
// Install: npm install --save-dev mocha chai

// math.test.js
const { expect } = require('chai');
const { add, subtract } = require('./math');

describe('Math functions', () => {
  it('should add two numbers', () => {
    expect(add(2, 3)).to.equal(5);
  });

  it('should subtract two numbers', () => {
    expect(subtract(10, 4)).to.equal(6);
  });
});
```

Test Runners

Configure your test script in `package.json`:

```
{
  "scripts": {
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage",
    "test:mocha": "mocha 'tests/**/*.test.js'",
    "test:verbose": "jest --verbose"
  }
}
```

Mocking Dependencies

```
// Jest mocking
jest.mock('../services/database');
const db = require('../services/database');
db.getUser.mockResolvedValue({ id: 1, name: 'Alice' });

test('getUserName returns user name', async () => {
  const result = await getUserName(1);
  expect(result).toBe('Alice');
  expect(db.getUser).toHaveBeenCalledTimes(1);
});
```

Integration Testing APIs

```
// Install: npm install --save-dev supertest

const request = require('supertest');
const app = require('../app');

describe('GET /api/users', () => {
  it('should return a list of users', async () => {
    const res = await request(app)
      .get('/api/users')
      .expect(200);

    expect(res.body).toBeInstanceOf(Array);
    expect(res.body[0]).toHaveProperty('name');
  });
});
```

Exercise: Write Jest tests for an Express.js task API. Test all CRUD endpoints (GET, POST, PUT, DELETE), including validation errors and 404 cases. Use supertest for HTTP integration testing.

Deployment

Environment Variables

Environment variables keep configuration separate from code. Never hard-code secrets.

```
# .env file (never commit this!)
PORT=3000
NODE_ENV=production
DATABASE_URL=mongodb://...
JWT_SECRET=your-secret-key
API_KEY=abc123

// Install: npm install dotenv
// Load in code
require('dotenv').config();

console.log(process.env.PORT);
console.log(process.env.NODE_ENV);
```

.env.example (commit this)

```
PORT=3000
NODE_ENV=development
DATABASE_URL=your-database-url
JWT_SECRET=your-secret
API_KEY=your-api-key
```

Development vs Production

Aspect	Development	Production
Error Detail	Full stack traces	Limited, user-friendly
Logging Level	Verbose (debug, info)	Minimal (warn, error)
Auto-reload	Enabled (nodemon)	Disabled
Caching	Minimal	Aggressive
Minification	No	Yes

CI/CD Pipeline

A typical CI/CD pipeline for Node.js applications includes linting, testing, building, and deploying.

```
# .github/workflows/deploy.yml
name: Node.js CI/CD

on:
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci
      - run: npm run lint
      - run: npm test
      - run: npm run build --if-present

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to production
        run: |
          # Deploy to your hosting provider
          echo "Deploying..."
```

Security Best Practices

- **Use HTTPS:** Never serve production over HTTP.
- **Validate Input:** Sanitize all user input to prevent injection attacks.
- **Rate Limiting:** Use `express-rate-limit` to prevent abuse.
- **Helmet:** Use the `helmet` middleware for security headers.
- **Dependencies:** Regularly audit with `npm audit`.
- **Secrets:** Never commit secrets. Use environment variables or secret manager.

- **CORS:** Restrict CORS to specific origins.

```
// Security setup example
const helmet = require('helmet');
const ratelimit = require('express-rate-limit');

app.use(helmet());
app.use(rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100,
  message: 'Too many requests, please try again later.'
}));
```

Deployment Strategies

- **Platform as a Service (PaaS):** Heroku, Render, Railway, Fly.io – quickest setup
- **Virtual Machines:** AWS EC2, DigitalOcean, Linode – full control
- **Containers:** Docker + Kubernetes – scalable, portable
- **Serverless:** AWS Lambda, Vercel, Netlify – auto-scaling, pay-per-use

```
# Example Dockerfile
FROM node:20-alpine

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .

EXPOSE 3000
CMD ["node", "src/index.js"]
```

Exercise: Create a Dockerfile for your Express.js task API. Add a .dockerignore file, multi-stage build, and use a non-root user. Then push it to a container registry and deploy to a cloud provider.

Performance & Scaling

Logging

Structured logging is essential for debugging and monitoring production applications.

```
// Install: npm install winston

const winston = require('winston');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});

// Usage
logger.info('Server started', { port: 3000 });
logger.error('Database connection failed', { error: err.message });
```

Monitoring

- **Process Managers:** PM2, forever – auto-restart, clustering
- **APM Tools:** New Relic, Datadog, Sentry – performance monitoring
- **Metrics:** Prometheus + Grafana – custom metrics dashboards
- **Health Checks:** Expose /health endpoint for load balancers

```
// Health check endpoint
app.get('/health', (req, res) => {
  const health = {
    uptime: process.uptime(),
    status: 'OK',
    timestamp: Date.now(),
    memory: process.memoryUsage()
  };
  res.json(health);
});

// Using PM2
// Install: npm install -g pm2
// pm2 start app.js -i max
// pm2 monit
// pm2 logs
// pm2 status
```

Performance Optimization

- **Use Gzip Compression:** `npm install compression`
- **Enable Caching:** Redis for in-memory caching
- **Optimize Database Queries:** Add indexes, avoid N+1 queries
- **Use Connection Pooling:** Reuse database connections
- **Minify and Bundle:** Reduce payload sizes
- **CDN:** Serve static assets via CDN

```
// Compression example
const compression = require('compression');
app.use(compression());

// Simple in-memory cache
const cache = new Map();
async function getCachedData(key, fetchFn, ttl = 60) {
  if (cache.has(key)) {
    const { data, expiry } = cache.get(key);
    if (Date.now() < expiry) return data;
    cache.delete(key);
  }
  const data = await fetchFn();
  cache.set(key, { data, expiry: Date.now() + ttl * 1000 });
  return data;
}
```

Child Processes

Use `child_process` to run CPU-intensive operations in separate processes.

```
const { spawn, exec, fork } = require('child_process');

// Spawn (stream output)
const ls = spawn('ls', ['-lh', '/usr']);
ls.stdout.on('data', (data) => console.log(`stdout: ${data}`));

// Exec (buffer output)
exec('node --version', (err, stdout, stderr) => {
  if (err) return console.error(err);
  console.log('Node version:', stdout);
});

// Fork (new Node.js process)
const child = fork('worker.js');
child.send({ task: 'compute' });
child.on('message', (result) => {
  console.log('Worker result:', result);
});
```

Clusters

The `cluster` module allows you to create child processes that share the same server port, utilizing all CPU cores.

```
const cluster = require('cluster');
const http = require('http');
const numCPUs = require('os').cpus().length;

if (cluster.isMaster) {
  console.log(`Master process ${process.pid} is running`);

  // Fork workers
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }

  cluster.on('exit', (worker, code, signal) => {
    console.log(`Worker ${worker.process.pid} died. Restarting...`);
    cluster.fork(); // Auto-restart
  });
} else {
  // Workers share the HTTP server
  http.createServer((req, res) => {
    res.writeHead(200);
    res.end(`Handled by worker ${process.pid}`);
  }).listen(3000);

  console.log(`Worker ${process.pid} started`);
}
```

Worker Threads

For CPU-intensive JavaScript operations, use `worker_threads` which share the same process but run on separate threads.

```
// main.js
const { Worker } = require('worker_threads');

const worker = new Worker('./worker-thread.js', {
  workerData: { iterations: 1000000 }
});

worker.on('message', (result) => {
  console.log('Result from worker:', result);
});

worker.on('error', (err) => console.error(err));

// worker-thread.js
const { parentPort, workerData } = require('worker_threads');

function compute(iterations) {
  let result = 0;
  for (let i = 0; i < iterations; i++) {
    result += Math.sqrt(i);
  }
  return result;
}

parentPort.postMessage(compute(workerData.iterations));
```

Exercise: Create a simple Express app with a CPU-intensive endpoint. Use PM2 to run it in cluster mode with max instances. Compare the performance before and after clustering using `autocannon` or `wrk`.

Advanced Topics

Microservices

Microservices architecture breaks an application into small, independent services that communicate over a network. Node.js is ideal for microservices due to its lightweight nature.

```
// Service 1: User Service (port 3001)
const express = require('express');
const app = express();
app.get('/api/users/:id', (req, res) => {
  res.json({ id: req.params.id, name: 'Alice' });
});
app.listen(3001);

// Service 2: Order Service (port 3002)
const express = require('express');
const app = express();
app.get('/api/orders', async (req, res) => {
  // Call User Service
  const userRes = await fetch('http://user-service:3001/api/users/1');
  const user = await userRes.json();
  res.json({ user, orders: [{ id: 1, total: 29.99 }] });
});
app.listen(3002);
```

Communication Patterns

- **HTTP/REST:** Simple, synchronous
- **Message Queues:** RabbitMQ, Kafka (async, decoupled)
- **gRPC:** High-performance, binary protocol
- **Event Bus:** Pub/sub pattern

WebAssembly (Wasm)

Node.js can execute WebAssembly modules for near-native performance in compute-heavy tasks.

```
// Load and run WebAssembly
const fs = require('fs');
const wasmBuffer = fs.readFileSync('module.wasm');

WebAssembly.instantiate(wasmBuffer).then(({ instance }) => {
  const result = instance.exports.add(5, 3);
  console.log('Wasm result:', result); // 8
});

// Write WebAssembly (WAT format)
// (module
//   (func $add (param $a i32) (param $b i32) (result i32)
//     local.get $a
//     local.get $b
//     i32.add)
//   (export "add" (func $add))
// )
```

HTTP/2

Node.js supports HTTP/2 natively, offering multiplexing, header compression, and server push.

```

const http2 = require('http2');
const fs = require('fs');

const server = http2.createServer({
  key: fs.readFileSync('server.key'),
  cert: fs.readFileSync('server.cert')
});

server.on('stream', (stream, headers) => {
  stream.respond({
    'content-type': 'text/plain',
    ':status': 200
  });
  stream.end('Hello, HTTP/2!\n');
});

server.listen(3000);

```

Performance Hooks

Use `perf_hooks` to measure performance in your Node.js applications.

```

const { performance, PerformanceObserver } = require('perf_hooks');

// Measure execution time
performance.mark('start');
// ... some operation ...
performance.mark('end');
performance.measure('Operation', 'start', 'end');

const obs = new PerformanceObserver((list) => {
  console.log(list.getEntries());
});
obs.observe({ entryTypes: ['measure'] });

```

VM Module

The `vm` module allows running JavaScript code in a sandboxed context.

```
const vm = require('vm');

const code = 'console.log("Hello from VM!"); return 42;';
const result = vm.runInNewContext(code);
console.log('Result:', result);

// Context with globals
const sandbox = { x: 10, y: 20 };
vm.createContext(sandbox);
vm.runInContext('result = x + y;', sandbox);
console.log(sandbox.result); // 30
```

TLS/SSL

Secure your Node.js server with TLS.

```
const https = require('https');
const fs = require('fs');

const options = {
  key: fs.readFileSync('private-key.pem'),
  cert: fs.readFileSync('certificate.pem'),
  ca: fs.readFileSync('ca-certificate.pem')
};

https.createServer(options, (req, res) => {
  res.writeHead(200);
  res.end('Secure connection!\n');
}).listen(443);
```

Zlib (Compression)

```
const zlib = require('zlib');
const fs = require('fs');

// Compress file
const gzip = zlib.createGzip();
const readStream = fs.createReadStream('input.txt');
const writeStream = fs.createWriteStream('input.txt.gz');
readStream.pipe(gzip).pipe(writeStream);

// Decompress
const gunzip = zlib.createGunzip();
fs.createReadStream('input.txt.gz')
  .pipe(gunzip)
  .pipe(fs.createWriteStream('output.txt'));
```

Real-World Examples

- **E-commerce Platform:** Express.js + MongoDB + Redis caching + Stripe payments
- **Real-time Dashboard:** Socket.IO + PostgreSQL + Chart.js
- **File Processing API:** Express.js + Multer (file upload) + Sharp (image processing)
- **Web Scraper:** Puppeteer + Cheerio + MySQL

Exercise: Research and write a brief proposal for splitting a monolithic Express.js application into microservices. Identify boundaries (e.g., users, orders, products), choose a communication pattern, and outline the API contracts.

Hardware & IoT with Node.js

Overview

Node.js can interact with hardware, making it a great choice for IoT (Internet of Things) applications. The combination of JavaScript's ease of use and Node.js's event-driven model works well for sensor data collection, device control, and real-time dashboards.

Raspberry Pi Setup

The Raspberry Pi is the most popular board for Node.js IoT projects.

```
# Install Node.js on Raspberry Pi (Raspberry Pi OS)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Verify
node --version
npm --version
```

GPIO (General Purpose Input/Output)

GPIO pins allow you to read sensors and control actuators. Use the `onoff` or `pigpio` library.

```

// Install: npm install onoff

const Gpio = require('onoff').Gpio;

// Setup LED on GPIO 17
const led = new Gpio(17, 'out');

// Setup button on GPIO 18
const button = new Gpio(18, 'in', 'both');

// Blink LED
let state = 0;
setInterval(() => {
  state = state ? 0 : 1;
  led.writeSync(state);
}, 1000);

// Read button
button.watch((err, value) => {
  if (err) return console.error(err);
  console.log('Button pressed! Value:', value);
});

// Cleanup on exit
process.on('SIGINT', () => {
  led.unexport();
  button.unexport();
  process.exit();
});

```

LED Projects

Traffic Light Simulation

```
const Gpio = require('onoff').Gpio;

const red = new Gpio(17, 'out');
const yellow = new Gpio(27, 'out');
const green = new Gpio(22, 'out');

function setLights(r, y, g) {
  red.writeSync(r);
  yellow.writeSync(y);
  green.writeSync(g);
}

// Simulate traffic light cycle
async function trafficCycle() {
  setLights(1, 0, 0); // Red
  await sleep(3000);
  setLights(1, 1, 0); // Red + Yellow
  await sleep(1000);
  setLights(0, 0, 1); // Green
  await sleep(3000);
  setLights(0, 1, 0); // Yellow
  await sleep(1000);
  trafficCycle();
}

function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

trafficCycle();
```

Sensor Reading

```
// Temperature sensor (DS18B20 on Raspberry Pi)
const fs = require('fs');

const sensorPath = '/sys/bus/w1/devices/28-xxxx/w1_slave';

function readTemperature() {
  const data = fs.readFileSync(sensorPath, 'utf8');
  const match = data.match(/t=(-?\d+)/);
  if (match) {
    const tempC = parseInt(match[1]) / 1000;
    const tempF = tempC * 9 / 5 + 32;
    return { celsius: tempC, fahrenheit: tempF };
  }
  return null;
}

setInterval(() => {
  const temp = readTemperature();
  if (temp) {
    console.log(`Temperature: ${temp.celsius.toFixed(1)}°C / ${temp.fahrenheit.toFixed(1)}°F`);
  }
}, 5000);
```

WebSocket Integration

Combine hardware with WebSockets for real-time browser dashboards.

```

const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const Gpio = require('onoff').Gpio;

const app = express();
const server = http.createServer(app);
const io = new Server(server);

const sensor = new Gpio(4, 'in', 'both');

// Serve static files
app.use(express.static('public'));

io.on('connection', (socket) => {
  console.log('Dashboard connected');

  // Send sensor data when it changes
  sensor.watch((err, value) => {
    if (err) return;
    socket.emit('sensor', { value, timestamp: Date.now() });
  });

  socket.on('toggle-led', (state) => {
    const led = new Gpio(17, 'out');
    led.writeSync(state ? 1 : 0);
  });
});

server.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});

```

IoT Components Guide

Component	Purpose	GPIO Pin Type
LED	Visual output	Output (digital)
Button	User input	Input (digital)
DHT11/DHT22	Temperature & humidity	Input (digital)
HC-SR04	Distance / ultrasonic	Input/Output (digital)
Servo Motor	Precise movement	PWM output
Buzzer	Sound output	Output (digital/PWM)

Exercise: Create a weather station on a Raspberry Pi that reads temperature and humidity from a DHT11 sensor every 10 seconds and serves the data on a real-time web dashboard using Socket.IO. Include a button to toggle an LED.

Practice & Certification

Node.js Compiler

Node.js uses the V8 JavaScript engine, which features JIT (Just-In-Time) compilation. However, there are tools to compile Node.js applications into standalone executables.

```
# pkg - Package Node.js into executable
npm install -g pkg
pkg app.js --output myapp

# nexe - Create standalone executables
npm install -g nexe
nexe app.js --output myapp.exe

# Node SEA (Single Executable Applications) - experimental
# node --experimental-sea-config sea-config.json
```

Examples

REST API (Complete Example)

```
const express = require('express');
const app = express();
app.use(express.json());

let tasks = [];
let id = 1;

app.get('/tasks', (req, res) => res.json(tasks));

app.post('/tasks', (req, res) => {
  const { title } = req.body;
  if (!title) return res.status(400).json({ error: 'Title required' });
  const task = { id: id++, title, completed: false };
  tasks.push(task);
  res.status(201).json(task);
});

app.put('/tasks/:id', (req, res) => {
  const task = tasks.find(t => t.id === parseInt(req.params.id));
  if (!task) return res.status(404).json({ error: 'Task not found' });
  Object.assign(task, req.body);
  res.json(task);
});

app.delete('/tasks/:id', (req, res) => {
  tasks = tasks.filter(t => t.id !== parseInt(req.params.id));
  res.status(204).send();
});

app.listen(3000);
```

Exercises (All Chapters)

1. **Ch1:** Create a script that prints your name, the date, and the OS platform.
2. **Ch2:** Read 3 files in parallel, concatenate, and write to a new file.
3. **Ch3:** Create an NPM project with chalk and print colored text.

4. **Ch4:** Build a CLI tool that reads, compresses, hashes, and writes metadata.
5. **Ch5:** Create a TypeScript file with a Task interface and compile it.
6. **Ch6:** Build a task management REST API with Express.js.
7. **Ch7:** Create a product catalog API with Mongoose and aggregations.
8. **Ch8:** Build a real-time chat application with Socket.IO and rooms.
9. **Ch9:** Write Jest tests for the task API with supertest.
10. **Ch10:** Create a Dockerfile for the task API with multi-stage build.
11. **Ch11:** Run the task API in cluster mode with PM2 and benchmark.
12. **Ch12:** Propose a microservices architecture for an e-commerce app.
13. **Ch13:** Build a weather station on Raspberry Pi with WebSockets.
14. **Ch14:** Complete practice problems below.

Quiz

1. What is the event loop in Node.js?
2. What is the difference between CommonJS and ES Modules?
3. How does Node.js handle multiple concurrent requests with a single thread?
4. What are the phases of the event loop?
5. When would you use `worker_threads` vs `cluster`?
6. What is the purpose of the `next()` function in Express middleware?
7. How do you handle unhandled promise rejections?
8. What is the difference between `require` and `import`?
9. How does the `fs.promises` API differ from `fs` callbacks?
10. What is the purpose of `package-lock.json`?

Practice Problems

1. Write a function that recursively lists all files in a directory tree.
2. Create a command-line to-do app that persists tasks to a JSON file.
3. Implement a rate limiter middleware for Express.
4. Build a real-time stock price simulator using Socket.IO.
5. Create a simple key-value store that persists to disk (similar to Redis).
6. Implement a URL shortener service with Express and a database.
7. Write a file watcher that logs changes and triggers a build script.

Study Plan

Week	Topics	Goal
1	Ch1-2	Understand Node.js architecture and async programming
2	Ch3-4	Master modules, NPM, and core modules
3	Ch5-6	Build web applications with Express
4	Ch7-8	Integrate databases and real-time communication
5	Ch9-10	Test, debug, and deploy applications
6	Ch11-13	Scale, optimize, and explore advanced topics
7	Ch14-15	Practice projects and review

Interview Preparation

Common Interview Questions

1. Explain the Node.js event loop in detail.
2. How do you handle errors in async functions?
3. What is the difference between `process.nextTick`, `setImmediate`, and `setTimeout`?
4. How does Node.js handle file system operations?
5. Explain streams and buffers in Node.js.
6. How would you scale a Node.js application?
7. What are the security best practices for Node.js applications?
8. How do you manage environment-specific configuration?
9. Explain middleware in Express.js.
10. How do you debug a Node.js application in production?

Bootcamp Projects

- **Beginner:** Blog API with Express and file-based storage

- **Intermediate:** E-commerce API with MongoDB, JWT auth, and Stripe integration
- **Advanced:** Real-time collaborative document editor (like Google Docs) using Socket.IO and OT
- **Expert:** Microservices-based video streaming platform with gRPC, Redis, and RabbitMQ

Certification Path

Consider the following certifications to validate your Node.js skills:

- **OpenJS Node.js Application Developer (JSNAD):** Foundational knowledge
- **OpenJS Node.js Services Developer (JSNSP):** Focus on building services
- **AWS Certified Developer – Associate:** Node.js on AWS
- **Microsoft Certified: Azure Developer Associate:** Node.js on Azure

Final Exercise: Build a complete e-commerce REST API with the following features: user registration/login with JWT, product catalog with categories, shopping cart, order management, payment integration (Stripe mock), and admin dashboard endpoints. Write comprehensive tests and create a Docker Compose setup.

References

A quick reference guide for the most commonly used Node.js built-in modules, APIs, and utilities.

Built-in Modules Quick Reference

EventEmitter

The `events` module provides the `EventEmitter` class, which is the foundation for many Node.js APIs.

```
const EventEmitter = require('events');

class MyEmitter extends EventEmitter {}

const emitter = new MyEmitter();

// Register listeners
emitter.on('event', handler);           // Add listener
emitter.once('event', handler);        // Single-use listener
emitter.prependListener('event', fn);   // Add to beginning
emitter.removeListener('event', fn);    // Remove specific
emitter.removeAllListeners('event');    // Remove all

// Emit
emitter.emit('event', arg1, arg2);

// Utility
emitter.listenerCount('event');         // Count listeners
emitter.eventNames();                  // All registered events
emitter.setMaxListeners(n);            // Set max (default 10)
```

Cluster

```
const cluster = require('cluster');
const os = require('os');

if (cluster.isMaster) {
  const numWorkers = os.cpus().length;

  // Fork workers
  for (let i = 0; i < numWorkers; i++) {
    const worker = cluster.fork();
  }

  // Events
  cluster.on('fork', (worker) => { /* Worker forked */ });
  cluster.on('online', (worker) => { /* Worker is online */ });
  cluster.on('listening', (worker, address) => { /* Worker listening */ });
  cluster.on('exit', (worker, code, signal) => {
    console.log(`Worker ${worker.process.pid} died`);
    cluster.fork(); // Auto-restart
  });

  // Disconnect all workers
  cluster.disconnect();
} else {
  // Worker process - runs your app
}
```

Crypto APIs

```
const crypto = require('crypto');

// Hashing
crypto.createHash('sha256').update('data').digest('hex');
crypto.createHash('md5').update('data').digest('base64');

// HMAC
crypto.createHmac('sha256', 'secret').update('data').digest('hex');

// Encryption (AES-256-CBC)
const algorithm = 'aes-256-cbc';
const key = crypto.randomBytes(32);
const iv = crypto.randomBytes(16);

function encrypt(text) {
  const cipher = crypto.createCipheriv(algorithm, key, iv);
  let encrypted = cipher.update(text, 'utf8', 'hex');
  encrypted += cipher.final('hex');
  return { iv: iv.toString('hex'), encrypted };
}

function decrypt(encrypted) {
  const decipher = crypto.createDecipheriv(
    algorithm, key, Buffer.from(encrypted.iv, 'hex')
  );
  let decrypted = decipher.update(encrypted.encrypted, 'hex', 'utf8');
  decrypted += decipher.final('utf8');
  return decrypted;
}

// Random values
crypto.randomBytes(16).toString('hex');
crypto.randomUUID(); // Generates UUID v4

// Password hashing (PBKDF2)
crypto.pbkdf2Sync('password', 'salt', 100000, 64, 'sha512').toString('hex');
```

Socket APIs (Net)

```
const net = require('net');

// TCP Server
const server = net.createServer((socket) => {
  console.log('Client connected');

  socket.on('data', (data) => {
    console.log('Received:', data.toString());
    socket.write('Echo: ' + data);
  });

  socket.on('end', () => console.log('Client disconnected'));
});

server.listen(8000, () => console.log('TCP server on port 8000'));

// TCP Client
const client = net.createConnection({ port: 8000 }, () => {
  client.write('Hello from client!');
});

client.on('data', (data) => console.log('Server:', data.toString()));
```

File Streams

```
const fs = require('fs');

// Readable Stream
const readStream = fs.createReadStream('input.txt', {
  highWaterMark: 64 * 1024, // 64KB chunks
  encoding: 'utf8'
});

readStream.on('data', (chunk) => console.log('Chunk:', chunk.length));
readStream.on('end', () => console.log('Finished reading'));
readStream.on('error', (err) => console.error(err));

// Writable Stream
const writeStream = fs.createWriteStream('output.txt');
writeStream.write('First line\n');
writeStream.write('Second line\n');
writeStream.end('Last line\n');
writeStream.on('finish', () => console.log('Finished writing'));

// Transform Stream (example: uppercase transform)
const { Transform } = require('stream');
const upperCase = new Transform({
  transform(chunk, encoding, callback) {
    this.push(chunk.toString().toUpperCase());
    callback();
  }
});

fs.createReadStream('input.txt')
  .pipe(upperCase)
  .pipe(fs.createWriteStream('output-upper.txt'));
```

HTTP/HTTPS APIs

```
const http = require('http');
const https = require('https');

// HTTP Server
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ status: 'ok' }));
});

server.listen(3000);

// HTTP Client (GET)
http.get('http://api.example.com/data', (res) => {
  let data = '';
  res.on('data', chunk => data += chunk);
  res.on('end', () => console.log(JSON.parse(data)));
});

// HTTP Client (POST)
const postData = JSON.stringify({ name: 'Alice' });
const options = {
  hostname: 'api.example.com',
  port: 443,
  path: '/users',
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Content-Length': Buffer.byteLength(postData)
  }
};

const req = https.request(options, (res) => {
  let body = '';
  res.on('data', chunk => body += chunk);
  res.on('end', () => console.log(JSON.parse(body)));
});

req.on('error', (e) => console.error(e));
req.write(postData);
req.end();
```

Readline Interface

```
const readline = require('readline');

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Ask a question
rl.question('What is your name? ', (answer) => {
  console.log(`Hello, ${answer}!`);
  rl.close();
});

// Read file line by line
const rlFile = readline.createInterface({
  input: fs.createReadStream('data.txt'),
  crlfDelay: Infinity
});

rlFile.on('line', (line) => {
  console.log('Line:', line);
});

rlFile.on('close', () => {
  console.log('File reading complete');
});
```

Unit (assert)

```
const assert = require('assert');

assert.strictEqual(actual, expected); // === equality
assert.notStrictEqual(actual, expected); // !==
assert.deepEqual(actual, expected); // Deep equality
assert.ok(value); // Truthy check
assert.throws(fn, error); // Expects error
assert.rejects(promise, error); // Expects rejected promise
assert.doesNotReject(promise); // Expects fulfilled promise
```

Useful Resources

- **Official Documentation:** nodejs.org/docs
- **NPM Registry:** npmjs.com
- **Node.js GitHub:** github.com/nodejs/node
- **Express.js Guide:** expressjs.com
- **Node.js Best Practices:** [Node.js Best Practices repo](#)
- **Node.js Testing:** Jest (jestjs.io), Mocha (mochajs.org)

Congratulations! You've completed the Node.js Tutorial: A Complete Guide. Keep building, keep experimenting, and happy coding!